

Comparing the RISC-V ISA Profiles of Two XMSS C Implementations

Wrenna Robson*

April 2026

1 Context

Two independent C implementations of XMSS (RFC 8391) have been profiled on RISC-V 64 in companion reports:

- *RISC-V ISA Profile of the XMSS C Implementation* — our from-scratch C implementation in `impl/c/`, designed to be structurally portable to Jasmin (no VLAs, no heap allocation, no function pointers).
- *RISC-V ISA Profile of the Upstream XMSS Reference Implementation* — the authoritative reference at [XMSS/xmss-reference](#).

This report compares the two profiles side by side. Both were produced with the same toolchain (`riscv64-linux-gnu-gcc 13.3.0, -march=rv64gc -mabi=lp64d -O3`), the same classifier (a mnemonic lookup table generated from the `riscv-opcodes` database), and the same per-object-file and per-extension tabulation. Any differences between the two profiles are therefore attributable to differences in *source code*, not to toolchain or methodology.

2 Archives under comparison

2.1 What is *in* each archive

Component	<code>impl/c/libxmss.a</code>	<code>libxmss-ref.a</code>
SHA-256 / SHA-512 core	✓(bundled)	— (OpenSSL <code>libcrypto</code>)
SHAKE128 / SHAKE256	✓(bundled)	✓(bundled <code>fips202.c</code>)
WOTS+	✓	✓
L-tree	✓	✓(inside <code>xmss_commons</code>)
Treehash / BDS state	✓	✓(inside <code>xmss_core_fast</code>)
BDS state (de)serialisation	✓	— (in-memory only)
XMSS / XMSS-MT core	✓	✓
Top-level API	✓	✓
Parameter derivation	✓	✓

*With Claude (Anthropic), which co-authored the analysis tooling and report text.

2.2 Structural asymmetry: SHA-2

The decisive asymmetry is SHA-2. The reference’s `hash.c` calls `SHA256/SHA512` from OpenSSL; its SHA-2 compression code lives in `libcrypto` and is *not* present in the archive. Our implementation ships its own SHA-2 in pure C (`sha2_local.c`).

As a result, direct counts are not apples-to-apples. A like-for-like comparison is only possible by netting out our SHA-2 layer (or by measuring only the algorithm layer). Both views are shown below.

2.3 Module mapping

The two code bases carve the XMSS algorithm differently. Table 1 establishes the correspondence used in the per-module comparisons.

Table 1: Module mapping between the two implementations.

Role	impl/c/	Reference
SHA-2 core	<code>sha2_local.c.o</code>	(in <code>libcrypto</code>)
Keccak / SHAKE	<code>shake_local.c.o</code>	<code>fips202.o</code>
Hash wrappers	<code>xmss_hash.c.o</code>	<code>hash.o</code>
ADRS manipulation	<code>address.c.o</code>	<code>hash_address.o</code>
WOTS+	<code>wots.c.o</code>	<code>wots.o</code>
L-tree	<code>ltree.c.o</code>	
Treeshash	<code>treeshash.c.o</code>	<code>xmss_commons.o</code>
BDS state (runtime)	<code>bds.c.o</code>	
BDS serialisation	<code>bds_serialize.c.o</code>	(not present)
XMSS / XMSS-MT core	<code>xmss.c.o</code> , <code>xmss_mt.c.o</code>	<code>xmss_core_fast.o</code>
Top-level API	(inside <code>xmss.c.o</code>)	<code>xmss.o</code>
Parameters	<code>params.c.o</code>	<code>params.o</code>
Utilities	<code>utils.c.o</code>	<code>utils.o</code>

3 Headline numbers

Table 2 gives the top-level counts.

Table 2: Headline comparison with `-march=rv64gc`.

Metric	impl/c/	Reference	Reference – impl/c
Object files	13	9	−4
Total instructions	9562	7620	−1942
I	9505 (99.40%)	7541 (98.96%)	−1964
M	57 (0.60%)	79 (1.04%)	+22
Zbb (absent in rv64gc)	0	0	—
C-encoded ratio	48%	53%	+5 pp
Unique mnemonics	52	54	+2

On absolute count, the reference archive is 1942 instructions smaller. Almost all of that gap, 1953 instructions, is our `sha2_local.c.o`: once SHA-2 is netted out, the two archives are within

11 instructions of each other (7609 vs 7620). In other words, the archives are the same size up to the SHA-2 layer that the reference excludes by construction.

The extension surface is also nearly identical: both use only I + M, with A, F, D, Zba, Zbb, Zbs, Zicsr, and Zifencei absent from both.

4 Where the M instructions are

Both implementations fall well below 2% M usage, but the composition differs.

Table 3: M-extension mnemonic breakdown.

Mnemonic	impl/c/	Reference	Compiler emits it for
<code>mulw</code>	37	68	32-bit index $\times$ parameter
<code>mul</code>	17	6	64-bit offset / pointer arithmetic
<code>divuw</code>	3	1	parameter derivation (h/d)
<code>divu</code>	0	2	SHAKE absorb/squeeze offset (reference only)
<code>remu</code>	0	2	SHAKE block-length modulus (reference only)
Total	57	79	

The reference’s 22 extra M-instructions come from two differences:

- **SHAKE implementation style.** The reference’s `fips202.c` uses ordinary / and % arithmetic on byte counts in the absorb/squeeze loops (`inlen % rate, nblocks = outlen/rate`). Our SHAKE implementation works in fixed block counts with power-of-two-friendly arithmetic, so the compiler never emits `divu/remu`.
- **Index-multiplication density.** The reference uses `mulw` 68 times; we use it 37 times. The gap mostly lives in `xmss_core_fast.o` (59 M uses in the reference) versus the combined `xmss_mt.o` + `xmss.o` + `bds.o` (21 M uses in our code). The reference computes signature-segment offsets (`i * params->n, i * 2*n`) inline in many places; our code factors common offsets into locals and carries them through the loops, so the compiler re-uses a single register-materialised offset instead of re-multiplying.

Neither pattern indicates any structural dependence on M: an RV64I-only target would compile the same C, with shift-and-add expansions in place of `mulw` and `divu`.

5 Per-module comparison

Table 4 aligns the per-module counts using the mapping in Table 1.

Several module-level asymmetries stand out.

5.1 Parameters: 286 vs 1077

Our `params.o` is roughly a quarter the size of the reference’s. Both modules do the same job—map a wire-format OID to the numeric parameters ($n, w, h, d, \dots$)—but the reference uses a large `switch/if` cascade over all twelve standardised OIDs, emitting many compare-and-branch sequences, whereas our `params.c` uses a compact lookup table indexed by OID. The difference is entirely structural and is well understood as a C-level coding choice: a jump table is cheaper on RV64 than a linear search.

Table 4: Per-module instruction counts. The reference’s `xmss_commons.o` is compared against the combined total of our `ltree`, `treehash`, and `bds.o` (where that combination makes sense).

Role	impl/c/	Reference	Delta
SHA-2 core	1953	—	(delegated)
SHAKE / Keccak	1337	1259	−78
Hash wrappers	1082	684	−398
ADRS manipulation	99	29	−70
WOTS+	760	498	−262
L-tree + treehash + BDS runtime	1743	⊂ 3169	—
BDS serialisation	473	—	(not present)
XMSS / XMSS-MT core + commons	1782	3682	+1900
Top-level API	(in core)	362	—
Parameters	286	1077	+791
Utilities	47	29	−18
Total (archive-local)	9562	7620	−1942

5.2 Hash wrappers and ADRS: the cost of explicit serialisation

Our `xmss_hash.o` (1082) and `address.o` (99) together total 1181 instructions against the reference’s `hash.o` (684) + `hash_address.o` (29) = 713.

Our hash wrappers are deliberately more elaborate. The reference passes ADRS as `uint32_t addr[8]` and serialises it to bytes inside each hash wrapper via a shared `addr_to_bytes` helper. We pre-serialise ADRS into a stack buffer and pass a pointer, which (a) makes the hash wrappers match the shape of the Jasmin port we intend to write, and (b) gives the compiler a stable input layout so the PRF/F/H cores can be emitted with fixed operand sizes. The cost is visible: 400 more instructions in the hash wrappers. This is a deliberate trade that pays off in the Jasmin port rather than in this archive’s instruction count.

5.3 WOTS+: 760 vs 498

Our `wots.o` is 262 instructions larger. A portion of this comes from the address-by-pointer calling convention (every F call takes a pre-serialised ADRS; the reference passes `uint32_t[8]` and serialises inside F). The rest is that our WOTS+ chain routine explicitly checks chain-length invariants (`start + steps <= W-1`) and has a dedicated fast path for `steps == 0` that the reference lacks. These defensive checks are motivated by the Jasmin port, where out-of-range chain indices would manifest as subtle verification-side bugs.

5.4 XMSS core and BDS: the shape differs, the size matches

The reference bundles WOTS+ pk generation, L-tree, treehash, BDS state, auth-path computation, XMSS keygen/sign/verify, and XMSS-MT all into `xmss_core_fast.o` (3169 insns). We split these into separate compilation units: `bds.o` (1130), `treehash.o` (484), `ltree.o` (129), `xmss.o` (637), `xmss_mt.o` (1145), and `xmss_commons` logic inlined into callers. The combined total, 3525, is within 11% of the reference’s single file, and the difference is dominated by our additional `bds_serialize.o` (473 insns) for wire-format serialisation of BDS state—a feature the reference’s `xmss_core_fast.c` lacks entirely.

6 Zbb: what changes when bit-manip is enabled

Both implementations were re-profiled with `-march=rv64gc_zbb`. The extent of Zbb uptake differs sharply, and the mix reveals the structural SHA-2 asymmetry from §2.2.

Table 5: Zbb uptake with `-march=rv64gc_zbb`.

Metric	impl/c/	Reference	Delta
I (rv64gc)	9505	7541	
I (rv64gc_zbb)	9281	6982	
I removed	-224	-559	
Zbb (rv64gc_zbb)	164	132	
Total (rv64gc_zbb)	9502	7193	
Net instruction saving	-60	-427	

The reference achieves a $7\times$ larger net code-size reduction from Zbb than we do (427 vs 60 instructions), despite emitting 32 fewer Zbb instructions than we do. That inversion is explained by the mix:

Table 6: Zbb mnemonic mix with `-march=rv64gc_zbb`.

Mnemonic	impl/c/	Reference	Native RV64 cost it replaces	Where
<code>rolw</code>	41	0	3 insns (<code>srliw+slliw+or</code>)	32-bit SHA-2 rotation
<code>roriw</code>	17	0	3 insns (same)	32-bit SHA-2 rotation
<code>rev8</code>	19	0	multi-insn byte-swap	SHA-2 endianness
<code>rol</code>	17	38	3 insns (<code>srl+sll+or</code>)	64-bit Keccak rho
<code>rori</code>	27	20	3 insns (same)	64-bit Keccak rho
<code>andn</code>	28	51	2 insns (<code>not+and</code>)	Keccak chi, XMSS-MT masks
<code>maxu</code>	12	20	~ 3 insns (branch-based)	BDS height comparisons
<code>minu</code>	3	2	~ 3 insns (branch-based)	BDS/SHAKE min
<code>zext.h</code>	0	1	2 insns (<code>slli+srli</code>)	16-bit widening
Total	164	132		

Two observations frame the comparison:

1. **Our archive has 32-bit SHA-2 rotations to optimise (`rolw`, `roriw`, `rev8`);** the reference does not, because the reference’s SHA-2 lives in OpenSSL’s `libcrypto` and is not part of the analysed archive. The 77 Zbb instructions emitted in our `sha2_local.c.o` ($41\text{ rolw} + 17\text{ roriw} + 19\text{ rev8}$) have no reference counterpart in this analysis: whatever Zbb `libcrypto` does or does not use is invisible here.
2. **The larger net saving on the reference side is an archive-size effect, not a Keccak-shape effect.** An earlier draft of this section attributed the reference’s larger net saving to its Keccak rho step folding into `rol/rori` more idiomatically than ours. Direct measurement (hand-unrolling our `keccak_f1600` into the same straight-line form as the reference’s `fips202.c`, then reprofiling) disproved that: the permutation itself is comparably sized in both, and the unrolled form actually inflated our archive overall. The real source of the per-instruction asymmetry is elsewhere in the SHAKE module: we ship an incremental `_ctx_{init,absorb,finalize,squeeze}` API (driven by Jasmin-portability rules that forbid VLAs and discourage caller-allocated whole-message buffers), which the reference does not. The extra API surface raises our baseline; the reference’s smaller baseline makes any given Zbb saving look bigger in relative terms. See

`reports/jasmin-scope-hash-vs-upper.md` for the full design-tax accounting.

The key qualitative finding is shared: **both** implementations concentrate their Zbb uptake in the hash layer. In our archive $142/164 = 87\%$ of Zbb lands in `sha2_local` + `shake_local`; in the reference, $108/132 = 82\%$ lands in `fips202.o`. The algorithm layer, in both cases, sees only opportunistic `maxu/ minu/ andn` uses that do not affect correctness or performance materially.

7 Takeaways

Same ISA surface.

Both implementations, compiled with `rv64gc`, use only I + M. XMSS at the algorithm level is I-dominated and does not structurally require anything more. M usage is below 2% in both cases and comes entirely from compiler-emitted index/offset arithmetic.

Netting out SHA-2 equalises the archives.

The 1942-instruction gap between the two archives is almost exactly our bundled SHA-2 (`sha2_local.c.o`, 1953 insns). Everything else—hash wrappers, ADRS handling, WOTS+, BDS, XMSS—differs by at most a few hundred instructions per module.

Structural differences are design choices, not algorithm cost.

Our `params.o` is 791 insns smaller because we use a table; our hash and WOTS modules are several hundred insns larger because we pre-serialise ADRS and include defensive range checks. These differences reflect the Jasmin-port preparation of our implementation, not anything about the XMSS algorithm itself.

Zbb matters most where SHA-2 lives.

Our archive benefits from Zbb via 32-bit SHA-2 rotations and `rev8`; the reference benefits via 64-bit Keccak rotations. In both cases $\geq 82\%$ of Zbb uptake is in the hash layer. The algorithm layer's Zbb benefit is negligible. For a self-contained XMSS implementation (ours), Zbb trims about 60 instructions; for the reference-as-archived (SHA-2 opaque), it trims about 427 largely by improving Keccak. A reference build that included SHA-2 in source form would show a far larger absolute Zbb benefit, dominated by the SHA-2 compression core.

ISA guidance for a Jasmin RISC-V port is unchanged.

Both analyses point to `rv64im` as the minimum functional target and `rv64gc_zbb` as the optimisation target for the hash layer. Algorithm-layer Jasmin code can be written in portable RV64I + M without loss, and the hash layer is where ISA-specific work pays back.

8 Reproducing this report

This report is a synthesis of two independently generated reports. To regenerate the raw data:

```
# Our C implementation
(cd impl/c && make rv)
isa/scripts/analyse.sh      # → xmss_rv64_isa_profile.md
isa/scripts/compare_zbb.sh  # → rv64gc vs rv64gc_zbb comparison

# Upstream reference implementation
isa/scripts/build_reference_lib.sh rv64gc
```

```
isa/scripts/build_reference_lib.sh rv64gc_zbb
isa/scripts/analyse.sh isa/binaries/libxmss-ref-rv64gc.a \
                      isa/reports/xmss_rv64_isa_profile_reference.md
isa/scripts/analyse.sh isa/binaries/libxmss-ref-rv64gc_zbb.a \
                      /tmp/xmss_rv64_isa_profile_reference_zbb.md
```

The tables in this report transcribe numbers from those generated .md files and from the companion reports `xmss_rv64_isa_profile.tex` and `xmss_rv64_isa_profile_reference.tex`.